FinShield: A Generative-AI Framework for Automated Android APK Analysis and Risk Scoring in Banking Environments

Samarth Patil
CSE-AIML
Vishwakarma Institute of Technology
(Savitribai Phule Pune University)
Pune, India

Tejas Patil
CSE-AIML
Vishwakarma Institute of Technology
(Savitribai Phule Pune University)
Pune, India

Tanishka Phad
CSE-AIML
Vishwakarma Institute of Technology
(Savitribai Phule Pune University)
Pune, India

Swanandi Salunkhe
CSE-AIML
Vishwakarma Institute of Technology
(Savitribai Phule Pune University)
Pune, India

Abstract*—As mobile usage has increased, mobile banking has enabled new methods to spread malicious Android apps that mimic financial services. These applications can be used to intercept SMS messages, to use overlay windows to steal credentials, or to abuse the Android Accessibility Services, or to communicate with controlled infrastructure. Checking each suspicious APK manually is time-consuming and expensive and is not scalable for financial institutions. In this paper, we introduce FinShield (also known as BOI Sentinel AI), an automated system for analyzing the Android applications in a banking environment. The framework includes static analysis, dynamic analysis on the sandbox, threat enrichment, machine-learning classification and generative-AI reporting. APKs are analyzed with Androguard, YARA, QuarkEngine, MobSF, Frida, and network traffic monitoring and then enriched with threat intelligence sources like AbuseIPDB, MalwareBazaar, URLHaus, OpenPhish. FinShield employs a retrieval-augmented generation pipeline, where collected evidence is fed to six specialised language-model agents, which generate an investigation summary, threat classification, and a bank-specific action playbook, inspired by MITRE ATT&CK for Mobile, CAPEC and curated malware intelligence. SHAP values are used to explain the influence of individual features on the final risk estimate generated by the underlying XGBoost classifier, which is based on a structured representation of features. The system returns a result ranging from 0 to 100, and classifies each application into a possible operational category: safe, low risk, suspicious, or highly malicious. FinShield is designed to be integrated within the institution's own infrastructure to ensure APK, reports, and other sensitive artefacts are kept on-premise. Initial prototype outcomes indicate that this hybrid method can be beneficial for quicker and more consistent APK triage, but more testing on existing banking malware and independent datasets is needed.

***Index Terms*—Android Malware, APK Analysis, Banking Trojans, Generative AI, Retrieval-Augmented Generation, MITRE ATT&CK, XGBoost, SHAP, MobSF, QuarkEngine, Fraud Detection**

I. Introduction

Mobile payments and banking applications have become an integral part of India's digital financial system. As users have grown, attackers have been targeting banking customers more and more via fraudulent Android apps. These apps are usually sent through phishing emails, social media messages, unofficial app stores or file sharing, and mimic the look of legitimate banking apps but conduct malicious activity in the background. This can involve reading or forwarding one-time codes, presenting fake login screens, logging into official sites of financial institutions, tracking user events of access and use, collecting relevant device data, and transferring stolen data to command-and-control servers, all of which can ultimately lead to the theft of credentials, account takeover and unauthorised transactions and financial fraud.

Signatures, blacklists or manual review are too slow to catch the number and rapid growth of Android malware. A security analyst usually has to work with multiple tools: decompilers, manifest analysers, network monitors, dynamic instrumentation frameworks, together with the output of each of these tools.They are usually required to work with a number of different tools: decompilers, manifest analysers, network monitors and dynamic instrumentation frameworks, as well as the output of each of these tools. To tackle this, FinShield integrates all the layers of analysis into a single workflow. The system is able to obtain an APK and generate static and dynamic indicators, validates them against pertinent threat-intelligence sources, and creates an investigation report. Machine-learning models add an extra risk rating, while a local language model converts the technical evidence into something that security and fraud-response teams can respond to.

This work mainly has the following contributions:

- An automated workflow for analysing Android APKs, from upload through to report generation.
- A static-analysis layer covering bytecode inspection, YARA matching, permission analysis, and QuarkEngine behavioural rules.
- A dynamic sandbox built on MobSF with custom Frida instrumentation.
- Integration of multiple threat-intelligence feeds and MITRE ATT&CK for Mobile mappings for enrichment.
- A retrieval-augmented generation system grounded in security knowledge relevant to Indian banking operations.
- An XGBoost-based risk-scoring model with SHAP-based explanations of the machine-learning output.
- An intent-prediction and attack-journey module that correlates technical indicators with likely fraud objectives.
- An architecture suited to on-premise deployment, aimed at reducing data-residency and privacy concerns.

The remainder of the paper describes the system architecture, analysis components, scoring approach, evaluation results, deployment model, and limitations of the system.

II. Background and Related Work

In general, Android malware analysis is done in two ways: static and dynamic. Static analysis looks at application without running it, and can reveal permissions, API usage, embedded URLs, package metadata, strings, and see unusual patterns of code. Dynamic analysis, on the other hand, simulates the application in a controlled environment and monitors its behaviour during runtime, such as network activity, SMS access, filesystem access, and interaction with other Android services.

Androguard is the APK and DEX inspection framework that is used by FinShield in Python. YARA includes a collection of rules to match known malware signatures, and can be effective for flagging suspicious artefacts in a timely fashion but these signatures and isolated artefacts don't provide enough context. QuarkEngine also performs static analysis of the link between API calls, making it easier to detect and identify a suspicious set of API calls, as opposed to simply suspecting a certain API call or permission – for instance, a request for device ID followed by the delivery of that ID via an SMS-related API can indicate data collection and exfiltration. Even if class and method names are obfuscated, this sequence-based analysis is helpful. MobSF controls the execution of the application and gathers telemetry security data from the Android environment; FinShield extends this information with the use of scripts for Frida to monitor specific activities performed at runtime, e.g., SMS operations, creation of overlays, accessibility events, cryptographic operations, network activity.

The feature-based malware classifier is first trained with the DREBIN dataset. The findings were obtained with the XGBoost model, which is well suited to structured, sparse feature vectors of this type, and combined with the model and added to make predictions more interpretable – SHAP. When an investigator can also view which permissions, APIs, indicators or behaviours led to the numerical score, then that is most useful in a banking context.

As of late, the application of large language models in cybersecurity has garnered research interest for activities including malware analysis, cybersecurity summarisation, and threat-intelligence interpretation. However, it is not always suitable for financial institutions to send data to external cloud services associated with APKs. Therefore, FinShield is a locally hosted model, backed with retrieval from curated security documents.

III. System Architecture

Each service performs a different task, using the same web interface, with FinShield using a centralised FastAPI back-end for communication. The design enables for flexibility within the system to add or change individual components without impacting the other parts of the system. On submission of an APK, the backend will calculate the SHA-256 hash and determine if this APK has been previously analysed. New samples are added to MinIO, metadata and analysis status are kept in PostgreSQL and then sent to the analysis services. When resources in the system are enough, static analysis, dynamic analysis and indicator extraction can be executed in parallel.

A. Service Components

TABLE I
FinShield Microservice Architecture Components

Service (Port)	Main Responsibility
FastAPI (8000)	Workflow control, API access, storage
Static analysis (8010)	Androguard, YARA, and QuarkEngine processing
MobSF sandbox (8008)	Android emulator and dynamic analysis
Dynamic analysis (8011)	MobSF and Frida orchestration
Threat intelligence (8012)	IOC lookups and enrichment
RAG engine (8013)	Document retrieval and knowledge grounding
AI investigation engine (8014)	Generate multiple agent reports using AI investigation engine
Fraud intent engine (8015)	Determines if the input is fraud or genuine, and constructs the attack-journey from the intent
Risk-scoring service (8016)	XGBoost classification and SHAP explanation

Docker Compose is used to package the full deployment. Structured records are stored in PostgreSQL, APKs and generated reports are kept in MinIO, and document embeddings are stored in ChromaDB. Ollama serves as the local language-model endpoint. The microservice design also supports restricted deployments – for example, an institution operating in an air-gapped environment can maintain its own blocklists and knowledge documents while disabling external intelligence feeds.

IV. Static Analysis

The static-analysis service examines the APK without executing it, making this an inherently fast first stage that avoids the risks of running an unknown application directly.

A. Permission Analysis

The Android manifest is checked and the permissions that are checked are those associated with SMS, accessibility services, device administration, package installation and other sensitive capabilities. A permission without its own context isn't necessarily an indicator of harmful behavior because the device-management tools, communication apps and enterprise utilities also need access to sensitive permissions. Permissive, therefore, is the case of permissions used as support, rather than definitive. The SYSTEMALERTWINDOW permission is not included, since many other, not malicious, apps also use it, specifically floating widgets and accessibility tools, and if an app is suspected to be doing an overlay attack, this is confirmed by dynamic observation if possible.

B. API and String Analysis

The DEX files that are part of the application are examined for common APIs linked to application abuse, such as dynamic class loading, SMS sending, access to the accessibility services, access to the device identifier, reflection, cryptography and network communication. The service also is able to find suspicious strings, domains, package names, IP addresses, and URLs. An allowlist to exclude common advertising and analytics domains to minimize noise from commonly used third-party libraries.

C. YARA Matching

The APK is scanned for the following YARA rules: banking trojan, SMS interception, overlay-based credential theft, dynamic code loading, and other suspicious behaviours. The YARA results are stored as evidence and used for the final risk score, but cannot be evaluated on their own as they can also be matched by a common library or other harmless component.

D. QuarkEngine Analysis

Where a single permission or method is not a definitive indicator of malicious behaviour, QuarkEngine can identify sequences of API calls that may be a sign of malicious behaviour. For instance, a call to get a device identifier followed by a call to an SMS related API is a more suspicious sequence than either call individually. The number of matching rules, highest and average confidence scores, and the presence of SMS- or banking-related flags are recorded in FinShield. These signals are used in addition to, and not in place of, the other analysis techniques.

V. Dynamic Analysis

Behaviour that is loaded at runtime, hidden behind obfuscation, or triggered only under specific conditions can be missed by static inspection alone. FinShield therefore executes APKs within a virtualised Android environment managed by MobSF, which handles emulator interaction, application installation, execution, and report collection. FinShield's Frida-based instrumentation monitors a range of runtime behaviours, including:

- Reading, creation, and transmission of SMS.
- Accessibility-service actions.
- Creation of overlay windows.
- Telephony and device-information requests.
- Cryptographic operations.
- WebView navigation.
- Native-library loading.
- Filesystem access.
- Outbound network connections.
- Attempts to detect or evade the emulated environment.

Network traffic is captured with TcpDump and cross-referenced against indicators identified during static analysis, helping distinguish ordinary application traffic from connections to malicious or suspicious infrastructure.

The main dynamic-analysis signals are:

- `sms_intercepted`: whether SMS activity was observed.
- `accessibility_abuse`: whether suspicious accessibility actions were observed.
- `overlay_attack_detected`: whether an overlay was detected over a banking interface.
- `network_requests`: a list of all outbound connections.
- `c2_connection_count`: the number of connections matched to confirmed command-and-control infrastructure.

Dynamic results depend on the execution path taken during the sandbox run, so the absence of an observed event does not necessarily mean the corresponding behaviour is absent from the application.

VI. Threat Intelligence and MITRE Mapping

The threat-intelligence service (TIS) enriches indicators drawn from the APK and the sandbox run. The prototype integrates four external sources:

- AbuseIPDB for IP reputation.
- MalwareBazaar for malware and hash information.
- URLHaus for known malicious URLs and domains.
- OpenPhish for phishing-related URL intelligence.

A six-hour in-memory cache reduces the frequency of repeated OpenPhish requests. For deployments that must remain offline, external services can be disabled, with the system falling back to locally hosted intelligence and curated blocklists.

Where applicable, findings are mapped to MITRE ATT&CK for Mobile techniques, for example:

TABLE II
Mapping Findings to MITRE ATT&CK for Mobile Techniques

Technique	Interpretation in FinShield
T1412 – Capture SMS Messages	SMS interception observed during execution.
T1417 – Input Capture	Overlay-based input capture.
T1544 – Dynamic Code Loading	Suspicious dynamic code or payload loading.
T1426 – System Information Discovery	Collection of device identifiers.
T1430 – Location Tracking	Use of location-related capabilities.
T1582 – SMS Control	Suspicious SMS transmission or control.

This mapping gives reports a shared vocabulary and helps connect technical observations to established mitigation guidance.

VII. Retrieval-Augmented Knowledge Engine

The RAG service supplies background context to the investigation agents. The prototype indexes content from the following sources:

- MITRE ATT&CK for Mobile.
- CAPEC attack-pattern descriptions.
- CERT-In advisories.
- Curated reports on Android malware families.

Documents are chunked and converted into vector embeddings using the BAAI/bge-small-en-v1.5 model, with the resulting vectors stored in ChromaDB. When an agent finding is received, the RAG service retrieves semantically related passages and adds them to the agent's prompt as supporting context. This is intended to reduce unsubstantiated claims and standardise security terminology across reports. Retrieved content should be version-tagged and periodically reviewed, since outdated or poorly written source documents can degrade the quality of the generated reports.

VIII. Multi-Agent Investigation Engine

Rather than relying on a single prompt for report generation, FinShield uses six specialised agent roles.

A. Agent Responsibilities

- **Static Agent**: Interprets permissions, API calls, YARA matches, extracted strings, and QuarkEngine results.
- **Dynamic Agent**: Summarises runtime events, Frida observations, network activity, and sandbox behaviour.
- **Threat-Intelligence Agent**: Reviews IOC results and possible malware-family associations.
- **Knowledge Agent**: Retrieves relevant MITRE, CAPEC, and CERT-In details.
- **Central Analyst**: Consolidates the specialist outputs into a single investigation summary.
- **Action Recommender**: Converts the evidence into a prioritised action plan for bank teams.

The Action Recommender can assign tasks to SOC, Fraud, IT, Legal and Customer-Support teams and prioritise the recommendations through a P1-P4 scale.

Agents are to differentiate between collected evidence and direct observation from supporting evidence and what is still unknown, and the Central Analyst is explicitly instructed not to report a threat without supporting evidence.

The prototype runs a locally-hosted 3B model version of Llama 3.2 via Ollama. While long parts of report text are generated in plain text, short classification parts can be generated and validated individually, because the long structured JSON responses were unreliable on CPU-only systems. Language-model output is not considered to be a source of facts on its own – it is used only to organise and explain evidence provided by the other analysis services.

B. Fraud Intent and Attack-Journey Reconstruction

Technical indicators alone do not always reveal what an attacker is ultimately trying to achieve. FinShield therefore includes a fraud-intent detection service that assigns one of eight categories:

- Credential theft.
- OTP interception.
- Account takeover.
- Data exfiltration.
- Overlay attack.
- Device takeover.
- Fraudulent transaction.
- Benign application.

For the current implementation, the language-model prompt is provided when the confirmed indicators are present and a rule-based fallback is provided when the model is not present or its evidence is not available or an invalid response is provided. For instance, if there is suspicious accessibility activity along with SMS interception, this could be considered as indicative of account takeover. The system keeps track of the evidence for each prediction and calculates an evidence-based value of confidence for the prediction based on signals that are independently confirmed.

The journey-reconstruction module creates a directed graph that maps the most probable sequence of attack steps with nodes like "gain access," "display fake login screen," "capture credentials," "intercept OTP," and "complete account takeover. This graph is displayed in the Web interface with Cytoscape.js for interoperability amongst malware analysts, fraud investigators and incident responders. It is not a complete, historical record of the attacker's activity, but is rather a working hypothesis.

IX. Risk Scoring

A. Model Design

The risk-scoring service is based on an XGBoost classifier that was trained using a 215 dimensional feature representation extracted from Android malware data. These features encompass permissions, API calls, intent information, hardware features and network features. The output probability from the classifier is automatically translated into a risk score from 0–100.

B. Confidence-Aware Controls

If a machine-learning model is able to confidently detect a malicious application, just for requesting multiple sensitive permissions, then it's also able to do so for some legitimate application. FinShield reduces the impact of this by using score caps when there are no runtime or threat-intelligence indicators to support the static results. The Prototype complies with this policy:

- High-confidence evidence: no cap is applied.
- Moderate confidence without confirmation: the score is capped within the upper end of the suspicious range.

- Limited evidence without confirmation: the score is capped within the low-risk range.

These thresholds should be tuned against a representative validation set. Score caps are intended to complement sound model evaluation, threshold selection, and ongoing monitoring, not replace them.

C. Risk Categories

TABLE III
OPERATIONAL RISK CATEGORIES AND SUGGESTED RESPONSES

Score	Category	Suggested response
0–29	Safe	No immediate action
30–54	Low risk	Review or discuss with parent/guardian
55–74	Suspicious	Block, contain, or investigate
75–100	Highly malicious	Block and start incident response

These categories are default operations and should be customized to each bank's risk appetite and response operations.

D. Explainability

SHAP TreeExplainer is used on the XGBoost model to calculate the effect of each feature on the prediction, categorizing features into three classes: High risk, Low risk and Limited effect. The most influential features are present in the report so that analysts are able to learn the "how" and "why" behind the respective score, a crucial property when the score is used to inform decisions regarding customer protection, application blocking or regulatory reporting. Obfuscation isn't used as a maliciousness indicator because it is also present in many other legitimate applications that are obfuscated using ProGuard or R8, and it would be risky to assume that any obfuscation is an indicator of a malicious app to prevent systematic false positives.

E. FRONTEND and Investigation Report

The frontend is implemented using Vite, React, and Tailwind CSS, and provides four main views:

- **Dashboard**: Provides data on the number of samples analyzed, the distribution of scores and historical trends. Upload: This is to upload APK files with progress displayed as the upload is done.
- **Analysis Detail**: Shows the detailed analysis report, containing Static findings, Dynamic events, Threat intelligence results, MITRE mappings, SHAP explanations, Fraud intent and suggested actions. **History**: Search and review previous analyses and their risk category.

Reports are also available as structured JSON, so that they can be integrated with SIEM, SOC, ticketing or fraud-monitoring systems, via an API on the backend.

X. EVALUATION

A. Datasets

The DREBIN-215 feature representation is used for training the XGBoost model. To cover newer malware families, additional corpora such as AndroZoo, CICMalDroid, and the AMD corpus are applied. Labelling method, targeted Android versions and time periods of applications and malware families collected vary between these datasets; if results are reported, its limitations must be explicitly mentioned.

B. Classification Results

These measurements provide a summary of the prototype's performance for a held-out evaluation split.

TABLE IV
XGBOOST MODEL PROTOTYPE EVALUATION METRICS

Metric	Result
Precision	0.94
Recall	0.91
F1-score	0.92
ROC-AUC	0.97

The policy that takes the confidence into account shrunk the number of benign applications that were classified as the highest risk by approximately 60 percent compared to the uncapped scoring policy, and the QuarkEngine also lowered the number of false positives for applications requesting multiple sensitive permissions. Some of these results should be taken with a grain of salt: data splits may include correlated applications, repeated malware families, or even old behavioural patterns, that may give results that seem more positive for a particular data split than they would be in reality. More stringent evaluation would involve using i.i.d. and family-wise split.

C. Processing Time

The prototype was able to process a typical 5 MB APK in about 3–5 minutes on a server equipped with eight CPU cores and 16 GB of RAM. The static analysis, including the machine learning classification step, took less than 15 seconds and the classification step alone usually took less than 5 seconds. The dynamic execution time was around 60-90 seconds depending on application usage and emulator settings and the language-model stage was the most time consuming as it involved several local inference calls. Experimental tests showed that end-to-end latency was shortened with GPU acceleration. The actual latency may vary depending on the emulator configuration, model quantisation, document-retrieval latency, storage performance and number of runtime events that a particular APK produces.

There are lots of things to consider and do while setting up a deployment for security reasons.

The aim of FinShield is to be executed on the bank's infrastructure and intermediate artefacts, analysis results, reports and APK files will be stored on internal systems. External threat-intelligence requests are optional and can be turned off for deployments where outbound communication is not allowed. The dynamic-analysis environment should not be connected to any production banking systems and should not have access to any internal services, credentials, customer data or administrative network, except as allowed by firewall rules and independently monitored.

Recommended deployment practices include:

- Installing the analysis server in a dedicated security zone.
- Restricting inter-microservice communication to required ports only.
- Encrypting stored objects in MinIO and database backups.
- Implementing role-based access control for reports and APKs.
- Rotating threat-intelligence credentials regularly.
- Logging uploads, analyses, report access, and administrative changes for audit purposes.
- Keeping MobSF, Frida, Android images, containers, and language models up to date.
- Periodically reviewing RAG documents for accuracy and version changes.
- Retraining the classifier as new malware families emerge.
- Testing the sandbox against anti-analysis and emulator-detection techniques.

The framework is designed to minimise manual workload and not to replace human review for high impact decisions. The automated score is not a 'magic number' or an automatic trigger for actions that impact customers; it should be used as a starting point for investigation.

XI. Limitations

There are a number of limitations to this design. Firstly, behaviour that occurs only over an extended period of sandbox time, is conditionally triggered, or is environment conditional will not be detected by dynamic analysis. Second, the machine-learning model's reliability will depend on the data used in the training: the more malware families that have not been included in the training data, the less reliable the scores for that malware will be; and the more similar or imbalanced the test data is with the training data, the more 'confident' the model is likely to be. Thirdly, a small, locally hosted language model can sometimes output an incomplete explanation or fail to include relevant evidence; RAG can help to ground but not prevent generation errors. Fourth, the sources of threat intelligence can lack full coverage, rate limit, or be out of date, so the absence of a match does not mean they are safe to use, but rather that they are not currently known to be unsafe. Lastly, the MITRE mappings and thresholds for risks should be reviewed regularly because the Android APIs, banking techniques, and regulatory expectations keep changing.

XII. Future Work

Planned future work aims to improve both detection quality and operational usefulness, including:

- Adding rules targeting new Indian banking trojans.
- Training models on datasets spanning different time periods.
- Conducting family-wise evaluation to better measure generalisation.
- Clustering dynamic event streams to surface previously unseen behaviours.
- Introducing graph-based representations of API and permission relationships.
- Strengthening anti-emulation testing and sandbox diversity.
- Incorporating analyst feedback into the report-review process.
- Linking the platform to approved APK feeds and internal fraud-monitoring systems.
- Evaluating larger local language models as hardware allows.
- Calibrating output probabilities and formalising uncertainty estimates.

XIII. Conclusion

This paper has presented FinShield, an automated framework for analysing Android applications in banking environments. It combines static analysis, dynamic execution, threat-intelligence enrichment, machine-learning-based classification, retrieval-augmented generation, and structured incident-response recommendations, with the goal of connecting low-level APK evidence to operational explanations that security, fraud, and IT teams can act on. Running language-model inference locally and supporting on-premise deployment also helps address data-residency concerns. Prototype results suggest that feature-based classification, runtime monitoring, and sequence-aware rules together improve APK triage, though the reported performance still needs to be validated against independent, contemporary, and temporally disjoint malware collections. FinShield is best understood as a research and engineering tool whose value depends on continued dataset updates, rule maintenance, sandbox coverage, and human oversight.

References

[1] A. Desnos et al., "Androguard: Reverse engineering and malware analysis for Android applications," *GitHub Repository*, 2021.

[2] D. Arp, M. Spreitzenbarth, M. Hubner, H. Gascon, and K. Rieck, "DREBIN: Effective and explainable detection of Android malware in your pocket," in *Proc. NDSS*, 2014.

[3] S. M. Lundberg and S. I. Lee, "A unified approach to interpreting model predictions," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, pp. 4765–4774, 2017.

[4] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.

[5] MITRE ATT&CK for Mobile, "Mobile Threat Matrix Taxonomy," *MITRE Corporation*, 2023. [Online]. Available: https://attack.mitre.org/matrices/mobile/